

high-performance hash tables, and spill-to-disk infrastructure—were not specific to Presto. These same building blocks were needed by Spark, stream processing systems, and ML feature engineering pipelines.

Rather than implementing the same execution engine separately for every system, Meta extracted these reusable components into a standalone library. That library became Velox, a shared, high-performance execution engine that could power multiple data processing frameworks.

Why was Velox needed?

The fragmentation of data processing engines: As data workloads evolved from traditional SQL analytics to real-time stream processing and AI pipelines, the industry responded by building specialised execution engines such as Presto, Spark, Trino, Druid, ClickHouse, and DuckDB. While each engine targeted different use cases, they all implemented the same core execution primitives independently.

Every project built its own columnar in-memory data format, expression evaluation engine, hash tables for joins and aggregations, spill-to-disk infrastructure, parquet and ORC readers, SQL functions such as SUM(), COUNT(), string, date, and array operations.

This duplicated years of engineering effort across organisations. More importantly, every bug fix, optimisation, or hardware improvement had to be implemented separately in every engine, making innovation slow and expensive.

Inconsistent SQL semantics:

Independent implementations also led to inconsistent behaviour across engines. Even simple functions such as `substr()`, `round()`, or `date_trunc()` often produced different results depending on the execution engine. At Meta, engineers found multiple implementations of the same string function, each using different indexing rules and error-handling behaviour.

These semantic differences made migrating workloads difficult and undermined trust in analytical results. The same SQL query could produce different answers across platforms, forcing data teams to spend significant time debugging correctness issues instead of building applications.

The JVM performance ceiling:

Most large scale analytical engines, including Presto, Spark, and Hive, relied on the Java Virtual Machine (JVM). Although the JVM provides excellent portability and developer productivity, it introduces limitations for high-performance analytical execution:

- Garbage collection causes unpredictable latency during large queries.
- Java object overhead increases memory consumption and reduces cache efficiency.
- Limited control over memory management prevents advanced techniques such as memory arenas and efficient spill-to-disk.
- Access to modern CPU features like SIMD vector instructions (SSE, AVX, AVX2) is significantly more constrained than in native C++.

As datasets continued to grow, these limitations became a major barrier to improving query performance.

Velox architecture

Before diving into the architecture, it is essential to understand a critical design boundary: Velox is an execution backend library, NOT a standalone database server.

Velox deliberately does not include a SQL parser, query optimiser, distributed RPC scheduler, or cluster manager. It leaves logical planning and scheduling to host engines like Presto or Spark, which pass a serialised physical plan tree (`velox::core::PlanNode`) down to Velox worker nodes for ultra-fast native C++ execution.

Layer 1: Calling system (Integration layer)

- **What it does:** The host framework (Presto Coordinator or Spark Driver) acts as the brain. It parses your SQL query, optimises the execution plan, and assigns tasks across the cluster.
- **Handoff:** The calling system compiles the query into a fully optimised query plan (`velox::core::PlanNode` tree) and hands it down to Velox worker nodes for execution.

Layer 2: Execution framework and operators

- **Task/Pipeline execution framework:** Spawns non-blocking worker

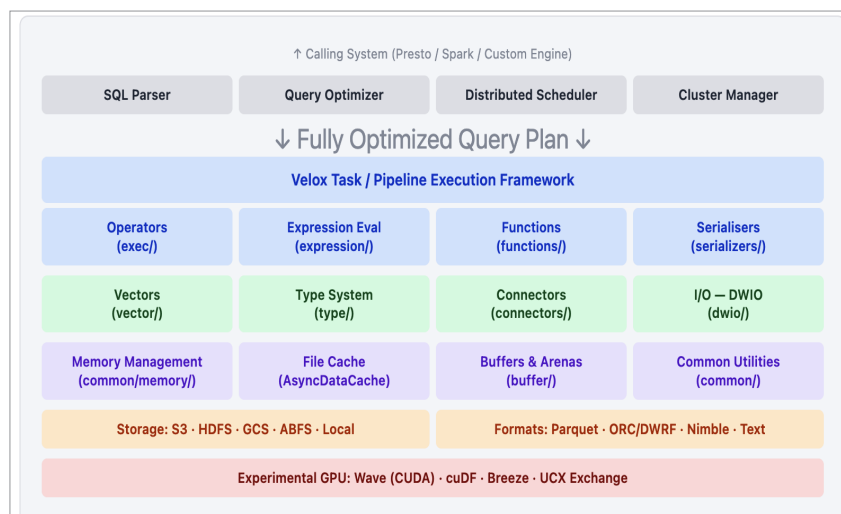


Figure 1: Architecture: How Velox is built